

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

ANUSHREE M (1BM25CS415)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by ANUSHREE M(1BM25CS415), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr. Namratha M
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	
2	Implement Johnson Trotter algorithm to generate permutations.	
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	
9	Implement Fractional Knapsack using Greedy technique.	
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	
11	Implement "N-Queens Problem" using Backtracking.	

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1: Write program to obtain the Topological ordering of vertices in a given digraph.

Program:

```
#include <stdio.h>

#define MAX 20

int main() {
    int n, adj[MAX][MAX];
    int indegree[MAX] = {0};
    int visited[MAX] = {0};
    int topo[MAX];
    int i, j, count = 0, k = 0, found;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            if(adj[i][j] == 1) {
                indegree[j]++;
            }
        }
    }
    while(count < n) {
        found = 0;
```

```

for(i = 0; i < n; i++) {
    if(indegree[i] == 0 && visited[i] == 0) {
        topo[k++] = i;
        visited[i] = 1;
        count++;
        found = 1;
        for(j = 0; j < n; j++) {
            if(adj[i][j] == 1) {
                indegree[j]--;
            }
        }
    }
}

if(found == 0) {
    break;
}

if(count != n) {
    printf("Cycle detected. Topological sorting not possible.");
}
else {
    printf("Topological Ordering: ");
    for(i = 0; i < k; i++) {
        printf("%d ", topo[i]);
    }
}

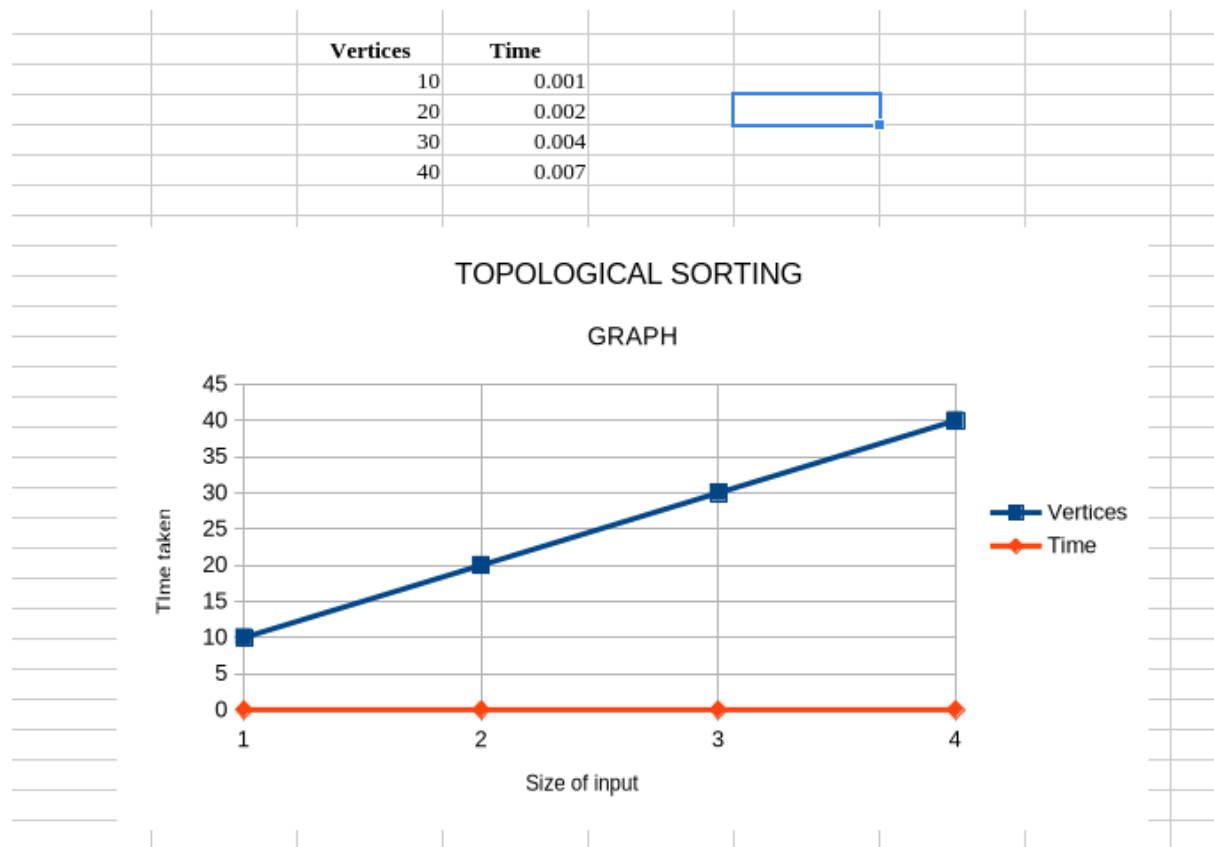
return 0;
}

```

Output:

```
ip@fedora:~/Documents/practice$ ./top
Enter number of vertices: 3
Enter adjacency matrix:
0 1 0
0 0 1
0 0 0
Topological Ordering: 0 1 2 ip@fedora:~/Documents/practice$
ip@fedora:~/Documents/practice$ ./top
Enter number of vertices: 3
Enter adjacency matrix:
0 1 0
0 0 1
0 0 1
❖ Cycle detected. Topological sorting not possible. ip@fedora:~/Documents/practice$
```

Graph



Leetcode Program:

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai.

Program:

```
#include <stdbool.h>

#include <stdlib.h>

bool dfs(int node, int* vis, int* pathVis,
         int** graph, int* graphSize) {
    vis[node] = 1;
    pathVis[node] = 1;
    for(int i = 0; i < graphSize[node]; i++) {
        int next = graph[node][i];
        if(!vis[next]) {
            if(dfs(next, vis, pathVis, graph, graphSize))
                return true;
        }
        else if(pathVis[next]) {
            return true;
        }
    }
    pathVis[node] = 0;
    return false;
}

bool canFinish(int numCourses,
               int** prerequisites,
               int prerequisitesSize,
               int* prerequisitesColSize) {
```

```

int** graph = malloc(numCourses * sizeof(int*));
int* graphSize = calloc(numCourses, sizeof(int));

for(int i = 0; i < numCourses; i++) {
    graph[i] = malloc(numCourses * sizeof(int));
}

for(int i = 0; i < prerequisitesSize; i++) {

    int a = prerequisites[i][0];
    int b = prerequisites[i][1];
    graph[b][graphSize[b]++] = a;
}

int* vis = calloc(numCourses, sizeof(int));
int* pathVis = calloc(numCourses, sizeof(int));
for(int i = 0; i < numCourses; i++) {
    if(!vis[i]) {
        if(dfs(i, vis, pathVis, graph, graphSize))
            return false;
    }
}

return true;
}

```


Output:

☑ Testcase | > Test Result

⌂ ^

Accepted

Runtime: 0 ms

☑ Case 1

☑ Case 2

Input

numCourses =

2

prerequisites =

[[1,0],[0,1]]

Output

false

Expected

false

Lab program 2: Implement Johnson Trotter algorithm to generate permutations.

Program:

```
#include<stdio.h>

int mobile(int a[], int dir[], int n)
{
    int mobile_prev = 0, mobile = 0;
    for(int i = 0; i < n; i++)
    {
        if(dir[a[i]-1] == 0 && i != 0)
        {
            if(a[i] > a[i-1] && a[i] > mobile_prev)
            {
                mobile = a[i];
                mobile_prev = mobile;
            }
        }
        if(dir[a[i]-1] == 1 && i != n-1)
        {
            if(a[i] > a[i+1] && a[i] > mobile_prev)
            {
                mobile = a[i];
                mobile_prev = mobile;
            }
        }
    }
    return mobile;
}

int search(int a[], int n, int mobile)
```

```

{
    for(int i = 0; i < n; i++)
    {
        if(a[i] == mobile)
            return i;
    }
    return 0;
}

void printPermutation(int a[], int n)
{
    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

void johnsonTrotter(int n)
{
    int a[n], dir[n];
    for(int i = 0; i < n; i++)
    {
        a[i] = i + 1;
        dir[i] = 0;
    }
    printPermutation(a, n);
    for(int i = 1; i < n; i++)
    {
        int fact = 1;
        for(int j = 1; j <= n; j++)
            fact *= j;
    }
}

```

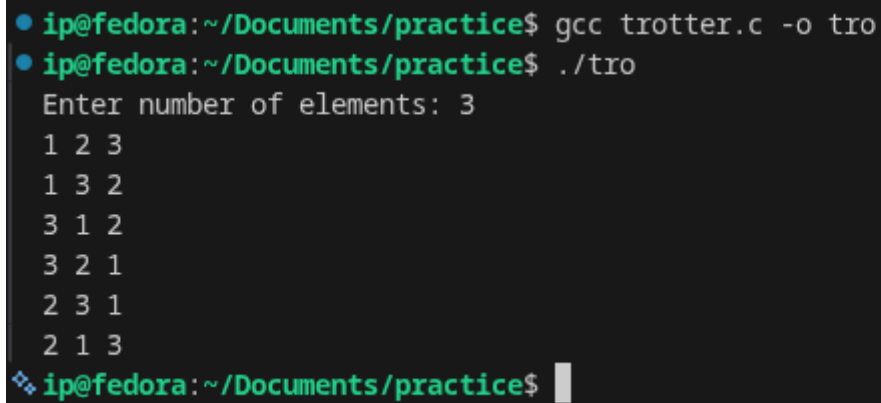
```

for(int k = 1; k < fact; k++)
{
    int mobileNum = mobile(a, dir, n);
    int pos = search(a, n, mobileNum);
    if(dir[mobileNum-1] == 0)
    {
        int temp = a[pos];
        a[pos] = a[pos-1];
        a[pos-1] = temp;
        pos = pos - 1;
    }
    else
    {
        int temp = a[pos];
        a[pos] = a[pos+1];
        a[pos+1] = temp;
        pos = pos + 1;
    }
    for(int j = 0; j < n; j++)
    {
        if(a[j] > mobileNum)
            dir[a[j]-1] = !dir[a[j]-1];
    }
    printPermutation(a, n);
}
break;
}
}

```

```
int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    johnsonTrotter(n);
    return 0;
}
```

Output:



```
ip@fedora:~/Documents/practice$ gcc trotter.c -o tro
ip@fedora:~/Documents/practice$ ./tro
Enter number of elements: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3
ip@fedora:~/Documents/practice$
```

Leetcode Program: Given an array nums of distinct integers, return all the possible permutations. You can return the answer in any order

Program:

```
#include <stdlib.h>

void swap(int* a, int* b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

void backtrack(int* nums, int numsSize, int start,
               int** result, int* returnSize,
               int* returnColumnSizes)
{
    if(start == numsSize)
    {
        result[*returnSize] = (int*)malloc(numsSize * sizeof(int));

        for(int i = 0; i < numsSize; i++)
            result[*returnSize][i] = nums[i];
        returnColumnSizes[*returnSize] = numsSize;
        (*returnSize)++;
        return;
    }
    for(int i = start; i < numsSize; i++)
    {
        swap(&nums[start], &nums[i]);
        backtrack(nums, numsSize, start + 1,
                  result, returnSize,
                  returnColumnSizes,
```

```

        returnColumnSizes);
    swap(&nums[start], &nums[i]);
}
}
int** permute(int* nums, int numsSize,
             int* returnSize,
             int** returnColumnSizes)
{
    int fact = 1;
    for(int i = 1; i <= numsSize; i++)
        fact *= i;
    int** result = (int**)malloc(fact * sizeof(int*));
    *returnColumnSizes =
        (int*)malloc(fact * sizeof(int));
    *returnSize = 0;
    backtrack(nums, numsSize, 0,
             result, returnSize,
             *returnColumnSizes);
    return result;
}

```

Output:

✓ Testcase | >_ Test Result

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

✓ Case 3

Input

```
nums =  
[1,2,3]
```

Output

```
[[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,2,1], [3,1,2]]
```

Expected

```
[[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]
```


Lab Program 3: Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

Program:

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

void merge(int a[], int l, int m, int r) {

    int i, j, k;

    int n1 = m - l + 1;

    int n2 = r - m;

    int L[n1], R[n2];

    for(i = 0; i < n1; i++)

        L[i] = a[l + i];

    for(j = 0; j < n2; j++)

        R[j] = a[m + 1 + j];

    i = 0;

    j = 0;

    k = l;

    while(i < n1 && j < n2) {

        if(L[i] <= R[j]) {

            a[k] = L[i];

            i++;

        } else {

            a[k] = R[j];

            j++;

        }

        k++;

    }

}
```

```

while(i < n1) {
    a[k] = L[i];
    i++;
    k++;
}
while(j < n2) {
    a[k] = R[j];
    j++;
    k++;
}
}

void mergesort(int a[], int l, int r) {
    if(l < r) {
        int m = (l + r) / 2;
        mergesort(a, l, m);
        mergesort(a, m + 1, r);
        merge(a, l, m, r);
    }
}

int main() {
    int n, i;
    clock_t start, end;
    double cpu_time;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    srand(time(0));
    for(i = 0; i < n; i++)
        a[i] = rand() % 1000;

```

```

start = clock();
mergesort(a, 0, n - 1);
end = clock();
cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Sorted elements are:\n");
for(i = 0; i < n; i++)
    printf("%d ", a[i]);
printf("\nTime taken = %f seconds\n", cpu_time);
return 0;
}

```

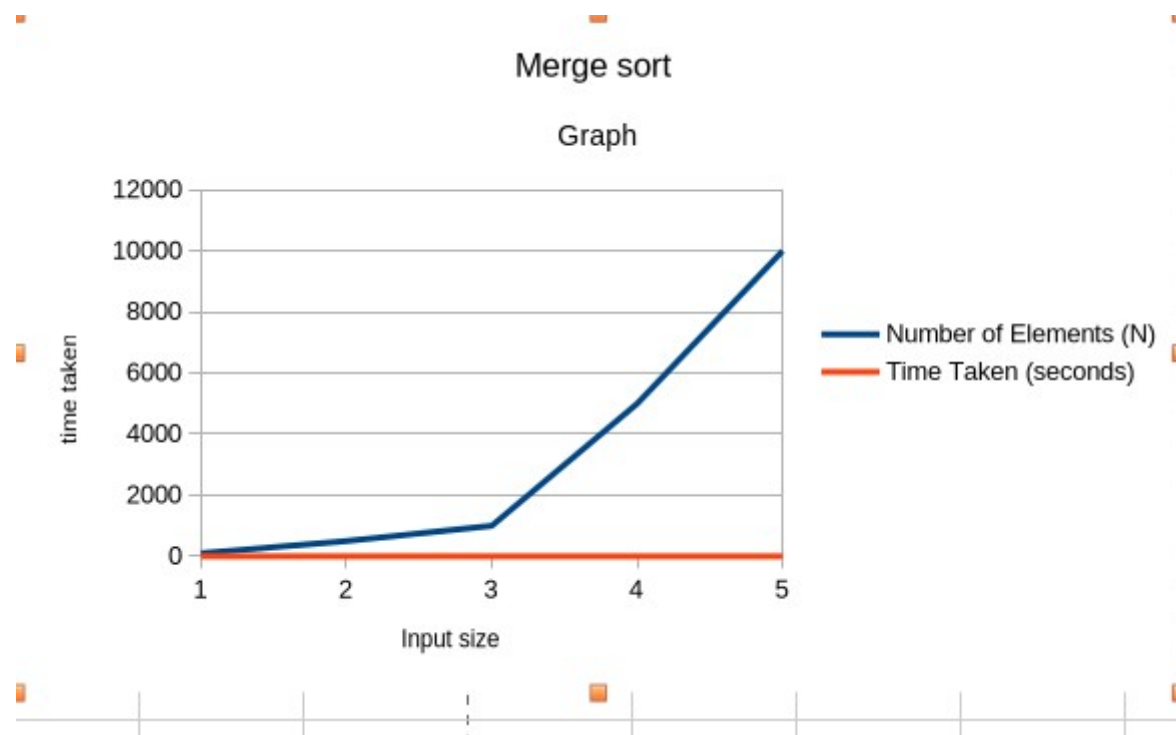
Output:

```

● ip@fedora:~/Documents/practice$ gcc merge.c -o merge
● ip@fedora:~/Documents/practice$ ./merge
Enter number of elements: 5
Sorted elements are:
65 226 378 516 952
Time taken = 0.000003 seconds
● ip@fedora:~/Documents/practice$ ./merge
Enter number of elements: 30
Sorted elements are:
0 4 8 33 38 143 274 281 311 471 497 582 636 654 657 672 678 702 759 765
Time taken = 0.000011 seconds
● ip@fedora:~/Documents/practice$ ./merge
Enter number of elements: 50
Sorted elements are:
5 41 72 83 91 101 121 123 141 214 268 286 330 333 343 356 426 488 506 506
6
Time taken = 0.000017 seconds
❖ ip@fedora:~/Documents/practice$

```

Graph:



Leetcode program: Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

Program:

```
void merge(int a[], int l, int m, int r){
    int i=l,j=m+1,k=0,temp[r-l+1];
    while(i<=m && j<=r){
        if(a[i]<a[j])
            temp[k++]=a[i++];
        else
            temp[k++]=a[j++];
    }
    while(i<=m)
        temp[k++]=a[i++];
```

```

while(j<=r)
    temp[k++]=a[j++];
for(i=l,k=0;i<=r;i++,k++)
    a[i]=temp[k];}
void mergesort(int a[], int l, int r){
    if(l<r){
        int m=(l+r)/2;
        mergesort(a,l,m);
        mergesort(a,m+1,r);
        merge(a,l,m,r);} }
int* sortArray(int* nums, int numsSize, int* returnSize) {
    mergesort(nums,0,numsSize-1);
    *returnSize=numsSize;
    return nums;
}

```

Output:

 Testcase |  Test Result

Accepted

Runtime: 0 ms

 Case 1

 Case 2

Input

nums =
[5,2,3,1]

Output

[1,2,3,5]

Expected

[1,2,3,5]

Lab Program 4: Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

Program:

```
#include<stdio.h>

#include<stdlib.h>

#include<time.h>

void quicksort(int a[],int low,int high){
    int i,j,pivot,temp;
    if(low<high){
        pivot=a[low];
        i=low;
        j=high;
        while(i<j){
            while(a[i]<=pivot&&i<high)
                i++;
            while(a[j]>pivot)
                j--;
            if(i<j){
                temp=a[i];
                a[i]=a[j];
                a[j]=temp;
            }
        }
        temp=a[low];
        a[low]=a[j];
        a[j]=temp;
        quicksort(a,low,j-1);
        quicksort(a,j+1,high);
    }
}
```

```

}

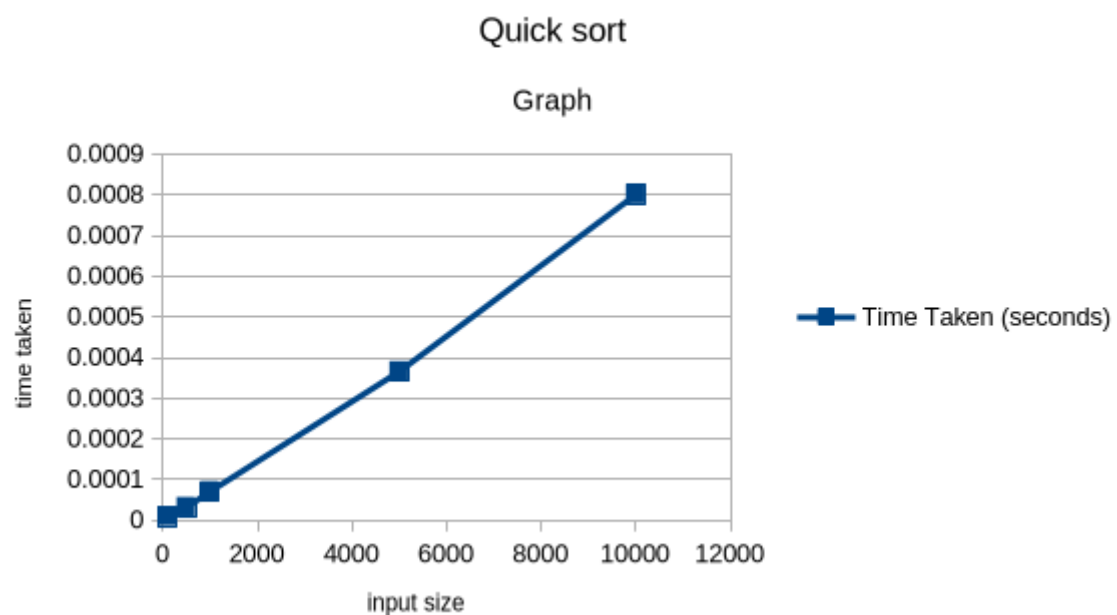
int main(){
    int n,i;
    clock_t start,end;
    double time_taken;
    printf("Enter number of elements: ");
    scanf("%d",&n);
    int a[n];
    srand(time(0));
    for(i=0;i<n;i++)
        a[i]=rand()%1000;
    start=clock();
    quicksort(a,0,n-1);
    end=clock();
    time_taken=((double)(end-start))/CLOCKS_PER_SEC;
    printf("Sorted elements are:\n");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);
    printf("\nTime taken = %f seconds\n",time_taken);
    return 0;
}

```

Output:

```
• ip@fedora:~/Documents/practice$ gcc quick.c -o quick
• ip@fedora:~/Documents/practice$ ./quick
Enter number of elements: 6
Sorted elements are:
242 397 413 432 638 984
Time taken = 0.000004 seconds
• ip@fedora:~/Documents/practice$ ./quick
Enter number of elements: 30
Sorted elements are:
144 162 187 205 206 328 338 340 348 388 498 527 540 544
Time taken = 0.000008 seconds
• ip@fedora:~/Documents/practice$ ./quick
Enter number of elements: 100
Sorted elements are:
4 10 13 23 42 58 65 70 114 135 138 152 169 211 225 231
71 484 509 513 520 528 535 558 561 567 573 574 580 585
90 996
Time taken = 0.000024 seconds
❖ ip@fedora:~/Documents/practice$
```

Graph:



Leetcode program: Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n\log(n))$ time complexity and with the smallest space complexity possible.

Program:

```
void quicksort(int a[],int low,int high){
    int i,j,pivot,temp;
    if(low<high){
        pivot=a[low];
        i=low;
        j=high;
        while(i<j){
            while(a[i]<=pivot && i<high)
                i++;
            while(a[j]>pivot)
                j--;
            if(i<j){
                temp=a[i];
                a[i]=a[j];
                a[j]=temp;
            }
        }
        temp=a[low];
        a[low]=a[j];
        a[j]=temp;
        quicksort(a,low,j-1);
        quicksort(a,j+1,high);}}

int* sortArray(int* nums, int numsSize, int* returnSize) {
    quicksort(nums,0,numsSize-1);
    *returnSize=numsSize;
```

```
    return nums;  
}
```

Output:

Accepted

Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

```
nums =  
[5,2,3,1]
```

Output

```
[1,2,3,5]
```

Expected

```
[1,2,3,5]
```

Lab Program 5: Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

Program:

```
#include<stdio.h>

#include<stdlib.h>

#include<time.h>

void heapify(int a[],int n,int i){
    int largest=i;
    int l=2*i+1;
    int r=2*i+2;
    int temp;
    if(l<n && a[l]>a[largest])
        largest=l;
    if(r<n && a[r]>a[largest])
        largest=r;
    if(largest!=i){
        temp=a[i];
        a[i]=a[largest];
        a[largest]=temp;
        heapify(a,n,largest);} }

void heapsort(int a[],int n){
    int i,temp;
    for(i=n/2-1;i>=0;i--)
        heapify(a,n,i);
    for(i=n-1;i>0;i--){
        temp=a[0];
        a[0]=a[i];
        a[i]=temp;
```

```

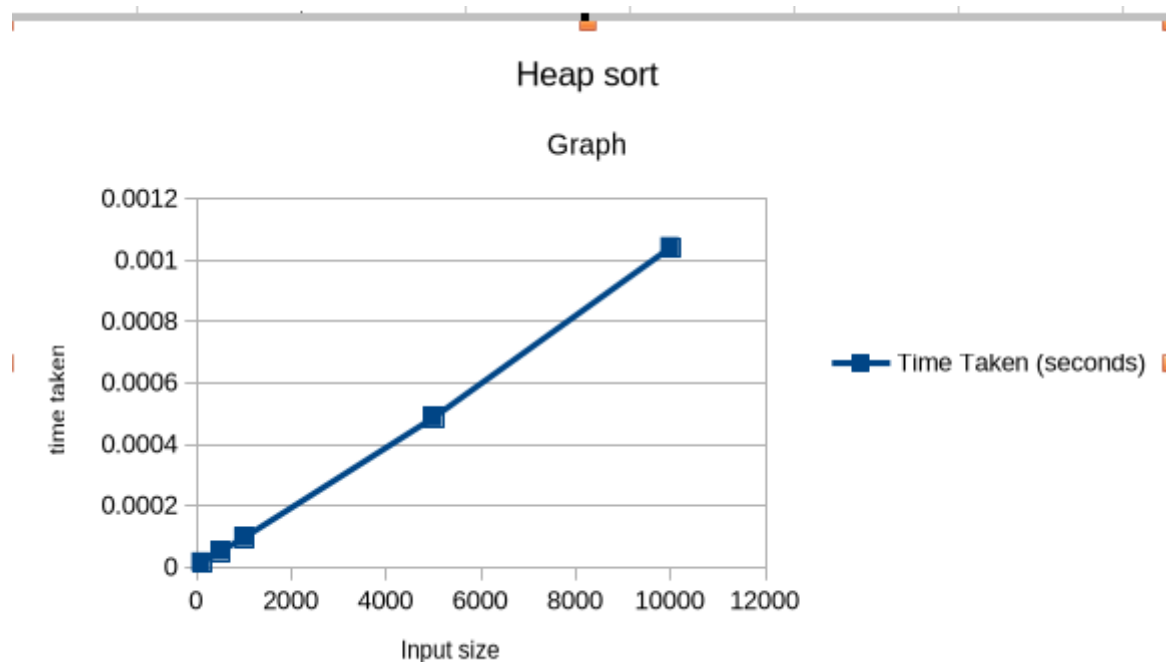
        heapify(a,i,0);}}
int main(){
    int n,i;
    clock_t start,end;
    double time_taken;
    printf("Enter number of elements: ");
    scanf("%d",&n);
    int a[n];
    srand(time(0));
    for(i=0;i<n;i++)
        a[i]=rand()%1000;
    start=clock();
    heapsort(a,n);
    end=clock();
    time_taken=((double)(end-start))/CLOCKS_PER_SEC;
    printf("Sorted elements are:\n");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);
    printf("\nTime taken = %f seconds\n",time_taken);
    return 0;
}

```

Output:

```
ip@fedora:~/Documents/practice$ gcc heap.c -o heap
ip@fedora:~/Documents/practice$ ./heap
Enter number of elements: 5
Sorted elements are:
158 350 584 626 870
Time taken = 0.000003 seconds
ip@fedora:~/Documents/practice$ ./heap
Enter number of elements: 50
Sorted elements are:
33 39 67 80 87 130 155 156 158 186 252 275 319 388 396
89
Time taken = 0.000015 seconds
ip@fedora:~/Documents/practice$ ./heap
Enter number of elements: 100
Sorted elements are:
3 4 5 12 12 19 25 66 79 80 85 119 130 132 137 157 173 2
3 555 576 579 581 591 594 598 609 629 636 637 638 645 6
9
Time taken = 0.000032 seconds
```

Graph:



Leetcode program: Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n\log(n))$ time complexity and with the smallest space complexity possible.

Program:

```
void heapify(int a[],int n,int i){
    int largest=i;
    int l=2*i+1;
    int r=2*i+2;
    int temp;
    if(l<n && a[l]>a[largest])
        largest=l;
    if(r<n && a[r]>a[largest])
        largest=r;
    if(largest!=i){
        temp=a[i];
        a[i]=a[largest];
        a[largest]=temp;
        heapify(a,n,largest);
    }
}

void heapsort(int a[],int n){
    int i,temp;
    for(i=n/2-1;i>=0;i--)
        heapify(a,n,i);
    for(i=n-1;i>0;i--){
        temp=a[0];
        a[0]=a[i];
        a[i]=temp;
        heapify(a,i,0);
    }
}
```

```
    }  
}  
int* sortArray(int* nums, int numsSize, int* returnSize) {  
    heapsort(nums,numsSize);  
    *returnSize=numsSize;  
    return nums;  
}
```

Output:

Accepted

Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

```
nums =  
[5,2,3,1]
```

Output

```
[1,2,3,5]
```

Expected

```
[1,2,3,5]
```

Lab Program 6: Implement 0/1 Knapsack problem using dynamic programming.

Program:

```
#include<stdio.h>

int max(int a,int b){
    if(a>b)
        return a;
    return b;
}

int main(){
    int n,m,i,w;
    printf("Enter number of items: ");
    scanf("%d",&n);
    int wt[n],val[n];
    printf("Enter weights:\n");
    for(i=0;i<n;i++)
        scanf("%d",&wt[i]);
    printf("Enter profits:\n");
    for(i=0;i<n;i++)
        scanf("%d",&val[i]);
    printf("Enter capacity of knapsack: ");
    scanf("%d",&m);
    int K[n+1][m+1];
    for(i=0;i<=n;i++){
        for(w=0;w<=m;w++){
            if(i==0 || w==0)
                K[i][w]=0;
            else if(wt[i-1]<=w)
                K[i][w]=max(val[i-1]+K[i-1][w-wt[i-1]],K[i-1][w]);
```



```

        else
            K[i][w]=K[i-1][w];
        }
    }
    printf("Maximum profit = %d",K[n][m]);
    return 0;
}

```

Output:

```

• ip@fedora:~/Documents/practice$ gcc knapsack.c -o knap
• ip@fedora:~/Documents/practice$ ./knap
Enter number of items: 3
Enter weights:
60
50
80
Enter profits:
50
90
80
Enter capacity of knapsack: 50
❖ Maximum profit = 90ip@fedora:~/Documents/practice$ 

```

Leetcode Program: Given an integer array nums, return true if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or false otherwise.

Program:

```

bool canPartition(int* nums, int numsSize) {
    int sum=0;
    for(int i=0;i<numsSize;i++)
        sum+=nums[i];
    if(sum%2)
        return false;
    int target=sum/2;
    bool dp[target+1];
}

```

```
for(int i=0;i<=target;i++)
    dp[i]=false;
dp[0]=true;
for(int i=0;i<numsSize;i++){
    for(int j=target;j>=nums[i];j--){
        dp[j]=dp[j]||dp[j-nums[i]];
    }
}
return dp[target];
}
```

Output:

☒ Testcase |  Test Result

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums =
[1,5,11,5]

Output

true

Expected

true

Lab Program 7: Implement All Pair Shortest paths problem using Floyd's algorithm.

Program:

```
#include<stdio.h>

int main(){
    int n,i,j,k;

    printf("Enter number of vertices: ");

    scanf("%d",&n);

    int a[n][n];

    printf("Enter cost matrix:\n");

    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            scanf("%d",&a[i][j]);
        }
    }

    for(k=0;k<n;k++){
        for(i=0;i<n;i++){
            for(j=0;j<n;j++){
                if(a[i][k]+a[k][j]<a[i][j])
                    a[i][j]=a[i][k]+a[k][j];} } }

    printf("All Pair Shortest Path Matrix:\n");

    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            printf("%d ",a[i][j]);

            printf("\n");
        }
    }

    return 0;
}
```

Output:

```
ip@fedora:~/Documents/practice$ gcc floyd.c -o floyd
ip@fedora:~/Documents/practice$ ./floyd
Enter number of vertices: 3
Enter cost matrix:
1 2 3
3 5 4
2 3 4
All Pair Shortest Path Matrix:
1
2
3
❖ ip@fedora:~/Documents/practice$
```

Leetcode Program 8: Find the City With the Smallest Number of Neighbors at a Threshold Distance

Program:

```
int findTheCity(int n, int** edges, int edgesSize, int* edgesColSize, int distanceThreshold) {
    int dist[101][101];
    int i,j,k;
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            if(i==j)
                dist[i][j]=0;
            else
                dist[i][j]=1000000;}}
    for(i=0;i<edgesSize;i++){
        int u=edges[i][0];
        int v=edges[i][1];
        int w=edges[i][2];
        dist[u][v]=w;
        dist[v][u]=w;}
```

```

for(k=0;k<n;k++){
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            if(dist[i][k]+dist[k][j]<dist[i][j])
                dist[i][j]=dist[i][k]+dist[k][j];}} }

int city=-1,minCount=1000000;

for(i=0;i<n;i++){
    int count=0;
    for(j=0;j<n;j++){
        if(dist[i][j]<=distanceThreshold)
            count++;}

    if(count<=minCount){
        minCount=count;
        city=i;}}

return city;}

```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

n =
4

edges =
[[0,1,3],[1,2,1],[1,3,4],[2,3,1]]

distanceThreshold =
4

Output

3

Expected

3

Lab Program 8: Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

Program:

```
#include<stdio.h>

int main(){
    int n,i,j,min,u,v,total=0;

    printf("Enter number of vertices: ");

    scanf("%d",&n);

    int cost[n][n],visited[n];

    printf("Enter cost adjacency matrix:\n");

    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            scanf("%d",&cost[i][j]);

            if(cost[i][j]==0)
                cost[i][j]=999;
        }
    }

    for(i=0;i<n;i++)
        visited[i]=0;

    visited[0]=1;

    printf("Edges in Minimum Spanning Tree:\n");

    for(int ne=1;ne<n;ne++){
        min=999;

        for(i=0;i<n;i++){
            if(visited[i]){
                for(j=0;j<n;j++){
                    if(!visited[j] && cost[i][j]<min){
                        min=cost[i][j];
                        u=i;
                    }
                }
            }
        }
    }
}
```

```

        v=j;}}}}

printf("%d -> %d = %d\n",u,v,min);

total+=min;

visited[v]=1;

}

printf("Minimum cost = %d",total);

return 0;

}

```

Output:

```

• ip@fedora:~/Documents/practice$ gcc prims.c -o prims
• ip@fedora:~/Documents/practice$ ./prims
Enter number of vertices: 4
Enter cost adjacency matrix:
1 2 1 3
2 1 2 1
1 1 1 2
3 2 1 2
Edges in Minimum Spanning Tree:
0 -> 2 = 1
2 -> 1 = 1
1 -> 3 = 1
Minimum cost = 3ip@fedora:~/Documents/practice$

```

Leetcode Program: You are given an array points representing integer coordinates of some points on a 2D-plane, where points[i] = [xi, yi]. The cost of connecting two points [xi, yi] and [xj, yj] is the manhattan distance between them: $|xi - xj| + |yi - yj|$, where $|val|$ denotes the absolute value of val. Return the minimum cost to make all points connected. All points are connected if there is exactly one simple path between any two points.

Program:

```

int minCostConnectPoints(int** points, int pointsSize, int* pointsColSize) {
    int visited[1000]={0},minDist[1000];
    for(int i=0;i<pointsSize;i++) minDist[i]=1000000;
    minDist[0]=0;
    int cost=0;

```

```
for(int i=0;i<pointsSize;i++){
    int u=-1;
    for(int j=0;j<pointsSize;j++){
        if(!visited[j]&&(u==-1||minDist[j]<minDist[u]))
            u=j;}
    visited[u]=1;
    cost+=minDist[u];
    for(int v=0;v<pointsSize;v++){
        int d=abs(points[u][0]-points[v][0])+abs(points[u][1]-points[v][1]);
        if(!visited[v]&&d<minDist[v])
            minDist[v]=d;}}
return cost;}
```

Output:

✓ Testcase | >_ Test Result

Accepted

Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

```
points =
[[0,0],[2,2],[3,10],[5,2],[7,0]]
```

Output

20

Expected

20

b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

Program:

```
#include<stdio.h>

#include<stdlib.h>

struct Edge{
    int u,v,w;
};

void swap(struct Edge *a,struct Edge *b){
    struct Edge t=*a;
    *a=*b;
    *b=t;
}

int parent[100];

int find(int i){
    while(parent[i]!=i)
        i=parent[i];
    return i;
}

void unionSet(int a,int b){
    parent[find(a)]=find(b);
}

int main(){
    int v,e,i,j;

    printf("Enter vertices and edges: ");

    scanf("%d%d",&v,&e);

    struct Edge edge[e];

    printf("Enter u v w:\n");

    for(i=0;i<e;i++)
```

```

scanf("%d%d%d",&edge[i].u,&edge[i].v,&edge[i].w);
for(i=0;i<v;i++)
    parent[i]=i;
for(i=0;i<e-1;i++){
    for(j=0;j<e-i-1;j++){
        if(edge[j].w>edge[j+1].w)
            swap(&edge[j],&edge[j+1]);} }
int cost=0;
printf("Edges in MST:\n");
for(i=0;i<e;i++){
    int a=find(edge[i].u);
    int b=find(edge[i].v);
    if(a!=b){
        printf("%d - %d : %d\n",edge[i].u,edge[i].v,edge[i].w);
        cost+=edge[i].w;
        unionSet(a,b);} }
printf("Minimum Cost = %d",cost);
return 0;
}

```

Output:

```

ip@fedora:~/Documents/practice$ gcc kurskal.c
ip@fedora:~/Documents/practice$ ./a.out
Enter vertices and edges: 4 5
Enter u v w:
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
Edges in MST:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10

```

Leetcode Program: You are given an array points representing integer coordinates of some points on a 2D-plane, where $\text{points}[i] = [x_i, y_i]$. The cost of connecting two points $[x_i, y_i]$ and $[x_j, y_j]$ is the manhattan distance between them: $|x_i - x_j| + |y_i - y_j|$, where $|val|$ denotes the absolute value of val. Return the minimum cost to make all points connected. All points are connected if there is exactly one simple path between any two points.

Program:

```
int p[1000];
int f(int x){
    return p[x]==x?x:(p[x]=f(p[x]));}
int cmp(const void *a,const void *b){
    return ((int*)a)[2]-((int*)b)[2];}
int minCostConnectPoints(int** points,int n,int* c){
    int e[100000][3],k=0;
    for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
            e[k][0]=i;
            e[k][1]=j;
            e[k][2]=abs(points[i][0]-points[j][0])+abs(points[i][1]-points[j][1]);
            k++;
        }
    }
    qsort(e,k,sizeof(e[0]),cmp);
    for(int i=0;i<n;i++) p[i]=i;
    int ans=0,cnt=0;
    for(int i=0;i<k&&cnt<n-1;i++){
        int a=f(e[i][0]),b=f(e[i][1]);
        if(a!=b){
            p[a]=b;
            ans+=e[i][2];
            cnt++;}
    }
    return ans;
}
```

Output:

☒ Testcase |  Test Result

Accepted

Runtime: 0 ms

☒ Case 1

☒ Case 2

Input

```
points =  
[[0,0],[2,2],[3,10],[5,2],[7,0]]
```

Output

20

Expected

20

Lab Program 9 : Implement Fractional Knapsack using Greedy technique.

Program:

```
#include<stdio.h>

struct Item{
    int weight,profit;
    float ratio;
};

int main(){
    int n,i,j,capacity;
    float total=0,temp;
    printf("Enter number of items: ");
    scanf("%d",&n);
    struct Item a[n];
    printf("Enter profit and weight:\n");
    for(i=0;i<n;i++){
        scanf("%d%d",&a[i].profit,&a[i].weight);
        a[i].ratio=(float)a[i].profit/a[i].weight;
    }
    printf("Enter capacity: ");
    scanf("%d",&capacity);
    for(i=0;i<n-1;i++){
        for(j=0;j<n-i-1;j++){
            if(a[j].ratio<a[j+1].ratio){
                struct Item t=a[j];
                a[j]=a[j+1];
                a[j+1]=t;}}
    }
    for(i=0;i<n;i++){
        if(a[i].weight<=capacity){
            total+=a[i].profit;
```

```

        capacity-=a[i].weight;
    }
    else{
        total+=a[i].ratio*capacity;
        break;}
}
printf("Maximum profit = %.2f",total);
return 0;
}

```

Output:

```

• ip@fedora:~/Documents/practice$ gcc fractknap.c -o fract
• ip@fedora:~/Documents/practice$ ./fract
Enter number of items: 3
Enter profit and weight:
60 10
100 20
120 30
Enter capacity: 50
❖ ip@fedora:~/Documents/practice$ █

```

•

Leetcode Program:It is a sweltering summer day, and a boy wants to buy some ice cream bars. At the store, there are n ice cream bars. You are given an array costs of length n, where costs[i] is the price of the ith ice cream bar in coins. The boy initially has coins coins to spend, and he wants to buy as many ice cream bars as possible.

Program:

```

int cmp(const void *a,const void *b){
    return *(int*)a-*(int*)b;
}

int maxIceCream(int* costs,int costsSize,int coins){
    qsort(costs,costsSize,sizeof(int),cmp);
    int c=0;

```

```
for(int i=0;i<costsSize;i++){  
    if(coins<costs[i]) break;  
    coins-=costs[i];  
    c++;  
}  
return c;  
}
```

Output:

✓ Testcase | >_ Test Result

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

✓ Case 3

Input

costs =
[1,3,2,4,1]

coins =
7

Output

4

Expected

4

Lab Program 10: From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

Program:

```
#include<stdio.h>

#define INF 9999

int main(){
    int n,i,j,u,v,count,min,start;

    printf("Enter number of vertices: ");
    scanf("%d",&n);
    int cost[n][n],dist[n],visited[n];
    printf("Enter adjacency matrix:\n")
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            scanf("%d",&cost[i][j]);
            if(cost[i][j]==0 && i!=j)
                cost[i][j]=INF;}}
    printf("Enter source vertex: ");
    scanf("%d",&start);
    for(i=0;i<n;i++){
        dist[i]=cost[start][i];
        visited[i]=0;}
    dist[start]=0;
    visited[start]=1;
    for(count=1;count<n-1;count++){
        min=INF;
        for(i=0;i<n;i++){
            if(dist[i]<min && !visited[i]){
                min=dist[i];
                u=i;}}
```

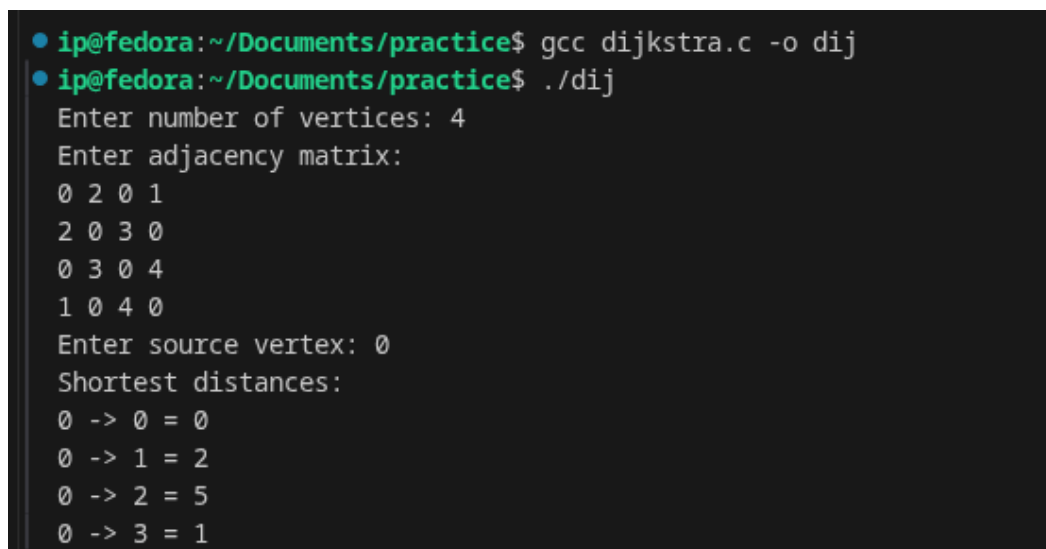


```

visited[u]=1;
for(v=0;v<n;v++){
    if(!visited[v] && dist[u]+cost[u][v]<dist[v]){
        dist[v]=dist[u]+cost[u][v];}}
printf("Shortest distances:\n");
for(i=0;i<n;i++){
    printf("%d -> %d = %d\n",start,i,dist[i]);
}
return 0;
}

```

Output:



```

ip@fedora:~/Documents/practice$ gcc dijkstra.c -o dij
ip@fedora:~/Documents/practice$ ./dij
Enter number of vertices: 4
Enter adjacency matrix:
0 2 0 1
2 0 3 0
0 3 0 4
1 0 4 0
Enter source vertex: 0
Shortest distances:
0 -> 0 = 0
0 -> 1 = 2
0 -> 2 = 5
0 -> 3 = 1

```

Leetcode Program:

You are given a network of n nodes, labeled from 1 to n . You are also given times, a list of travel times as directed edges $times[i] = (u_i, v_i, w_i)$, where u_i is the source node, v_i is the target node, and w_i is the time it takes for a signal to travel from source to target. We will send a signal from a given node k . Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1.

Program:

```
typedef struct{
    int node,dist;
}Pair;

int networkDelayTime(int** times,int timesSize,int* timesColSize,int n,int k){
    int g[101][101],d[101],v[101]={0};
    for(int i=1;i<=n;i++)
        for(int j=1;j<=n;j++)
            g[i][j]=1e9;
    for(int i=0;i<timesSize;i++){
        g[times[i][0]][times[i][1]]=times[i][2];
    }
    for(int i=1;i<=n;i++){
        d[i]=1e9;
    }
    d[k]=0;
    for(int c=1;c<=n;c++){
        int m=1e9,u=-1;
        for(int i=1;i<=n;i++){
            if(!v[i]&& d[i]<m){
                m=d[i];
                u=i;
            }
        }
        if(u==-1) break;
        v[u]=1;
        for(int j=1;j<=n;j++){
            if(d[u]+g[u][j]<d[j])
                d[j]=d[u]+g[u][j];
        }
    }
    int ans=0;
    for(int i=1;i<=n;i++){
        if(d[i]==1e9) return -1;
    }
}
```

```
        if(d[i]>ans) ans=d[i];
    }
    return ans;
}
```

Output:

Accepted

Runtime: 0 ms

✓ Case 1

✓ Case 2

✓ Case 3

Input

```
times =
[[2,1,1],[2,3,1],[3,4,1]]
```

```
n =
4
```

```
k =
2
```

Output

```
2
```

Expected

```
2
```

Lab Program 11: Implement “N-Queens Problem” using Backtracking.

Program:

```
#include<stdio.h>

#include<stdlib.h>

int x[20],n;

int place(int k,int i){
    for(int j=1;j<k;j++){
        if(x[j]==i||abs(x[j]-i)==abs(j-k))
            return 0;
    }
    return 1;
}

void queen(int k){
    for(int i=1;i<=n;i++){
        if(place(k,i)){
            x[k]=i;
            if(k==n){
                for(int j=1;j<=n;j++)
                    printf("%d ",x[j]);
                printf("\n");
            }
            else
                queen(k+1);
        }
    }
}

int main(){
    printf("Enter number of queens: ");
```

```

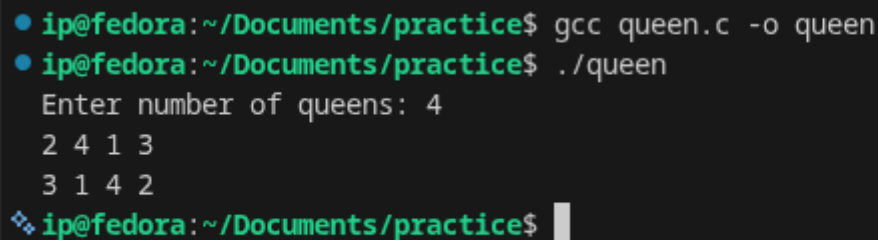
scanf("%d",&n);

queen(1);

return 0;
}

```

Output:



```

ip@fedora:~/Documents/practice$ gcc queen.c -o queen
ip@fedora:~/Documents/practice$ ./queen
Enter number of queens: 4
2 4 1 3
3 1 4 2
❖ ip@fedora:~/Documents/practice$

```

Leetcode Problem:

The n-queens puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other. Given an integer n, return all distinct solutions to the n-queens puzzle. You may return the answer in any order. Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Program:

```

int n;

char*** ans;

int sz;

int col[20],d1[40],d2[40];

char** makeBoard(int *pos){

    char **b=malloc(n*sizeof(char*));

    for(int i=0;i<n;i++){

        b[i]=malloc((n+1)*sizeof(char));

        for(int j=0;j<n;j++)

            b[i][j]='.';
    }
}

```

```

        b[i][pos[i]]='Q';
        b[i][n]='\0';}

    return b;
}

void f(int r,int *pos){
    if(r==n){
        ans[sz++]=makeBoard(pos);
        return;}

    for(int c=0;c<n;c++){
        if(!col[c]&&!d1[r-c+n]&&!d2[r+c]){
            col[c]=d1[r-c+n]=d2[r+c]=1;
            pos[r]=c;

            f(r+1,pos);

            col[c]=d1[r-c+n]=d2[r+c]=0;}} }

char*** solveNQueens(int x,int* returnSize,int** returnColumnSizes){
    n=x;
    sz=0;
    ans=malloc(1000*sizeof(char**));
    int *pos=malloc(n*sizeof(int));
    f(0,pos);
    *returnSize=sz;
    *returnColumnSizes=malloc(sz*sizeof(int));

    for(int i=0;i<sz;i++)
        (*returnColumnSizes)[i]=n;
    return ans;
}

```

Output:

✓ Testcase | >_ Test Result

Accepted

Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

n =

4

Output

[[".Q..", "...Q", "Q...", "..Q."], [".Q.", "Q...", "...Q", ".Q.."]]

Expected

[[".Q..", "...Q", "Q...", "..Q."], [".Q.", "Q...", "...Q", ".Q.."]]